

CS & IT ENGINEERING



COMPUTER ORGANIZATION AND ARCHITECTURE

Instruction & Addressing Modes

Lecture No.- 05

By- Vishvadeep Gothi sir



Recap of Previous Lecture



Doubts ?



Topic

Instruction Cycle

Topic

Fetch & Execution Cycle

Topic

Branch Instruction

Topics to be Covered



Topic

Addressing Modes

Topic

PC Relative Mode

Topic

Questions on Addressing Modes

Ans = 3

#Q. Consider a register-based architecture system (2-address instructions). For this system the following intermediate code is going to be converted in machine code. Minimum how many registers are required in system so that the code can run without register spill?

Consider first operand is always the register operand and it's the destination for operation too.

$t1 = X + Y$

$t2 = t1 - Z$

$t3 = t1 + t2$

$t4 = M + t3$

Assume X, Y, Z and M are memory operands

$R1 \leftarrow X$

$R2 \leftarrow Y$

$R1 \leftarrow R1 + R2$

$R2 \leftarrow R1$

$R3 \leftarrow Z$

$R1 \leftarrow R1 - R3$

$R2 \leftarrow R2 + R1$

$R1 \leftarrow M$

$R1 \leftarrow R1 + R2$



Topic : Addressing Modes

- It specifies how and from where the operands are obtained for an instruction

Add. mode $\longrightarrow$ E.A.
or
operand





Topic : Implied Mode

The opcode definition itself defines the operand

Opcode	Mode	Address
--------	------	---------

↓
Implied

operation
+
operand
↓
special purpose reg.

example:- INCA
↓
increment AC

explicit add. $\Rightarrow$ only for
GPR and
mem. operands

no any explicit add. needed
for special purpose reg.



Topic : Immediate Mode

The address field of instruction specifies the operand value

Opcode	Mode	Address
--------	------	---------

operand value

Usage:-

To use constant in instruction

$AC \leftarrow \underline{\#3}$
Immediate mode



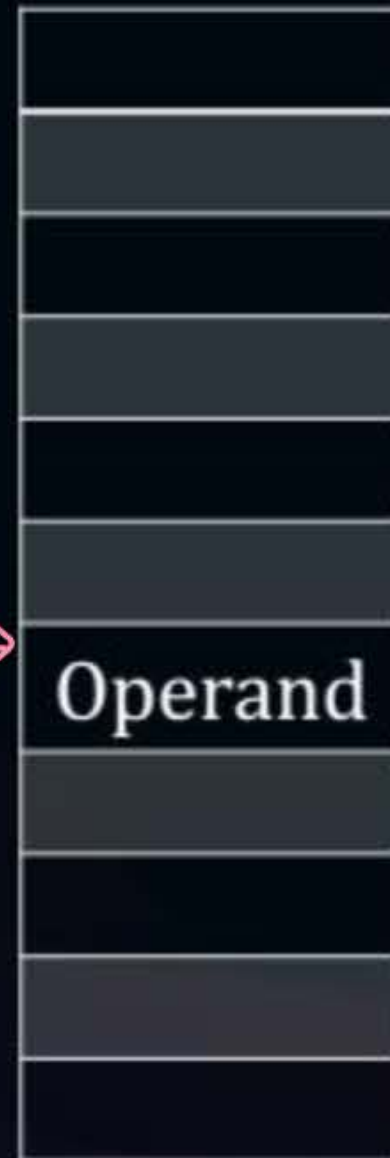
Topic : Direct Mode

absolute mode

The address field of instruction specifies the effective address



direct



Memory



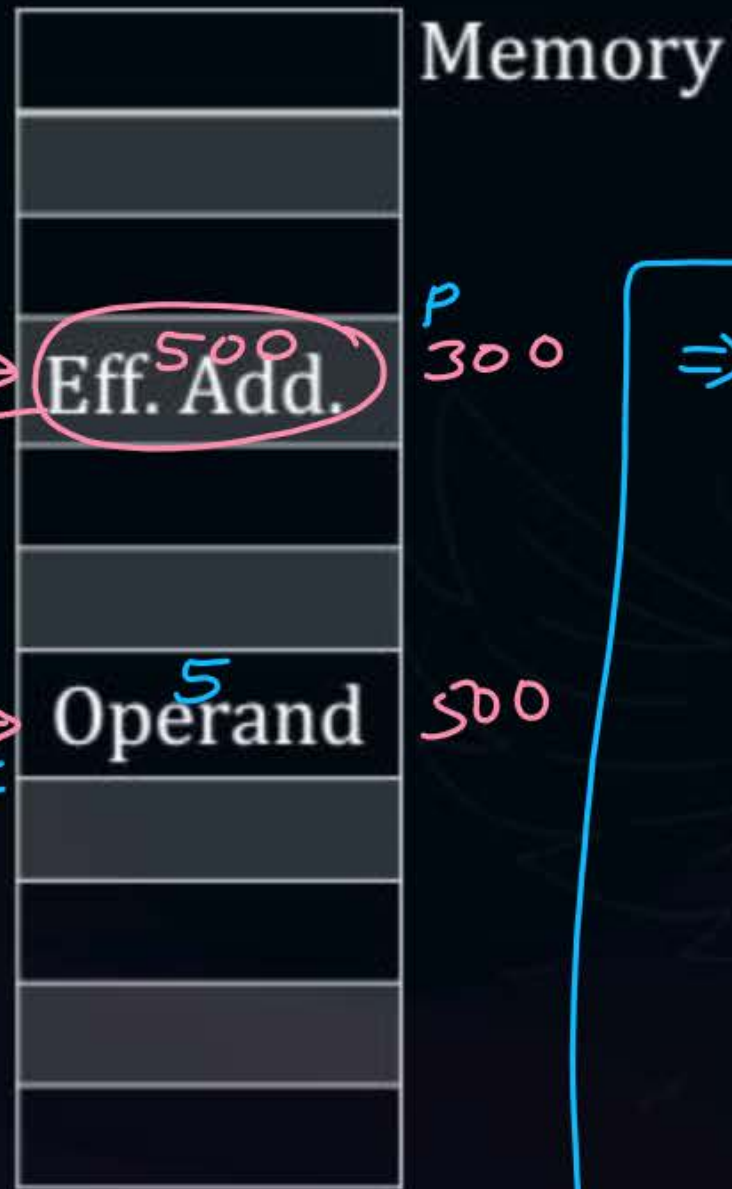
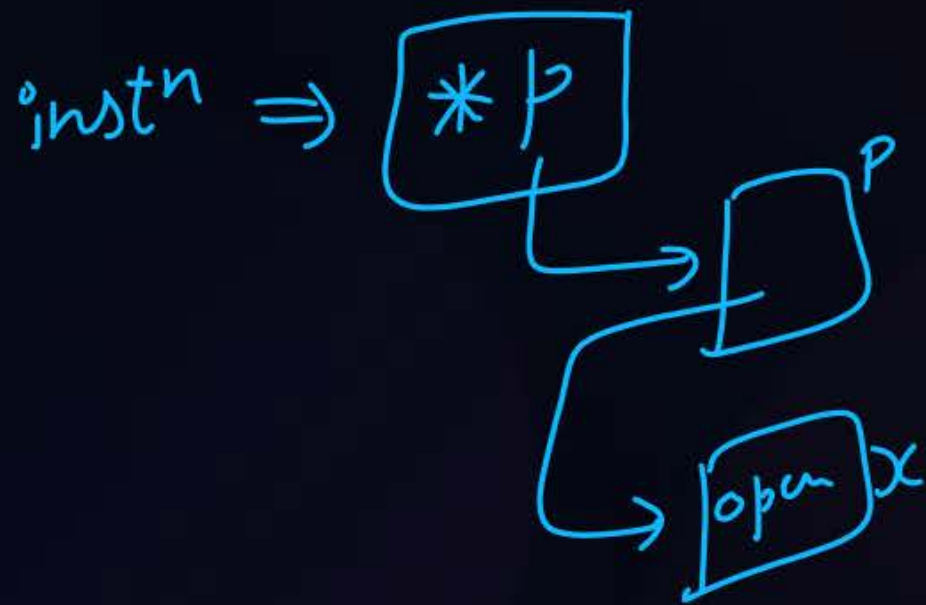
Topic : Indirect Mode

The address field of instruction specifies the address of effective address

Opcode	Mode	Address ³⁰⁰ *p
--------	------	---------------------------

ex:-

```
int x = 5;  
int *p = &x;
```



Usage:-

To implement pointers

⇒ To obtain operand,
2 mem. accesses needed.

assembly lang's -

ADD R1, @1000
↑ indirect mode



Topic : Register Mode



(Reg. direct mode)

The address field of instruction specifies a register which holds operand

Opcode	Mode	Address
--------	------	---------



operand
Register



ex:-

1. $R1 \leftarrow R1 + R2$

both operands are using
Reg. mode

2. $R1 \leftarrow \underbrace{R1}_{\text{Reg. mode}} * \underbrace{\#5}_{\text{Imm. mode}}$ $\Rightarrow$

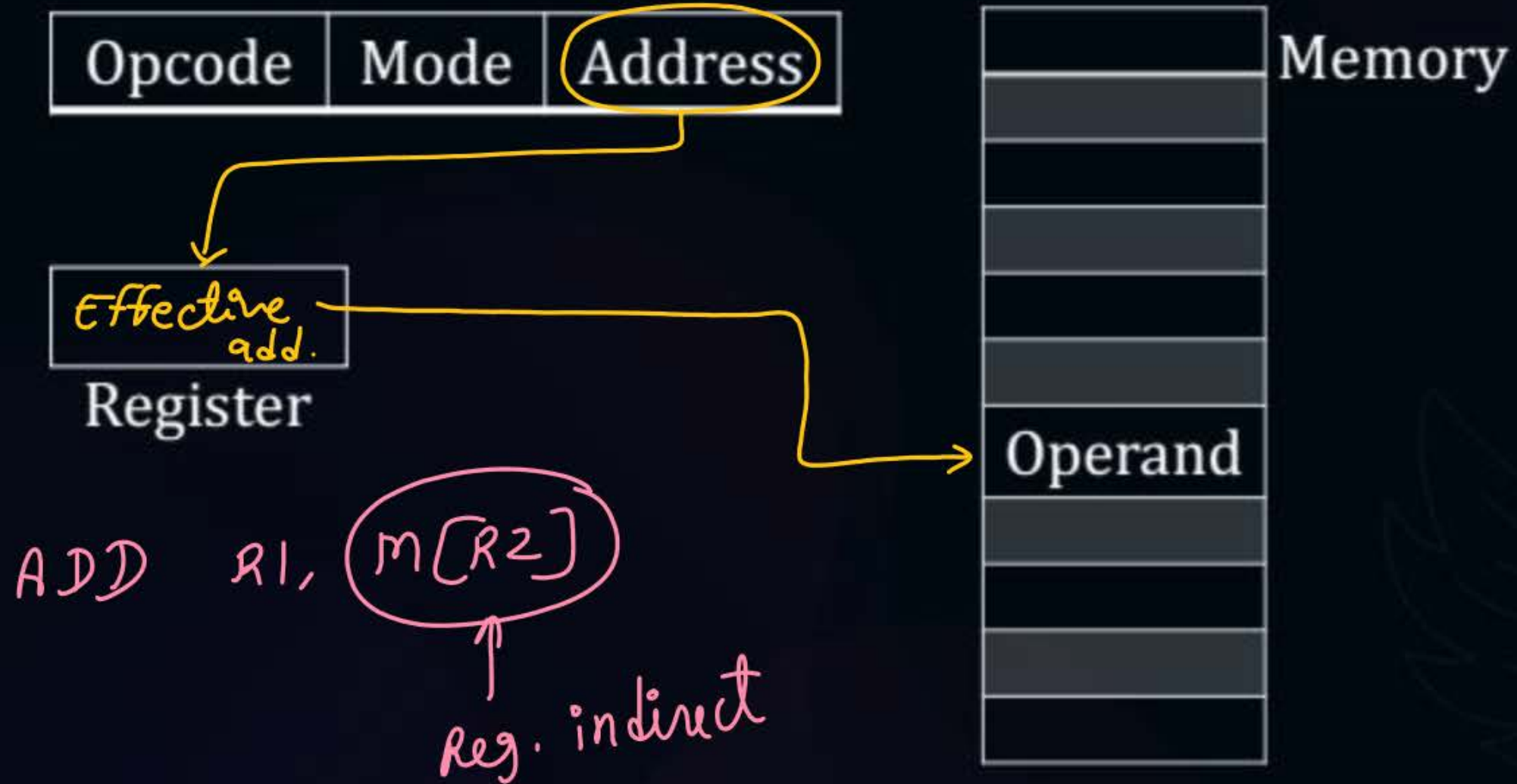
MUL R1, #5

3. $\underbrace{R2}_{\text{Reg. mode}} \leftarrow \underbrace{M[1000]}_{\text{Direct mode}}$



Topic : Register Indirect Mode

The address field of instruction specifies a register which holds effective address



Usage :-

To reduce (shorten) the instruction length.

Assume System RAM = 4GB = $2^{32}_B \Rightarrow \text{add.} = 32 \text{ bits}$
no. of GPRs = 128 = $2^7 \Rightarrow \text{Reg. number} = 7 \text{ bits}$

Direct



$\Rightarrow (x + y + 32) \text{ bits}$

Time to obtain operand

= 1 mem. access

Reg. Indirect



$\Rightarrow (x + y + 7) \text{ bits}$

= 1 reg. access + 1 mem. access



Topic : Autoincrement/Autodecrement Mode

Variant of register indirect mode, in which content of register is automatically incremented or decremented to access a table of content sequentially.



Autoinc.

500 504 508

Register
R3

500
304



Memory

Amount of increment or decrement depends on size of data (operand)

4 possibilities =>

→ pre inc. mode

→ post inc. mode

→ pre dec. mode

→ post dec. mode

=> default

=> default

} old microprocessors

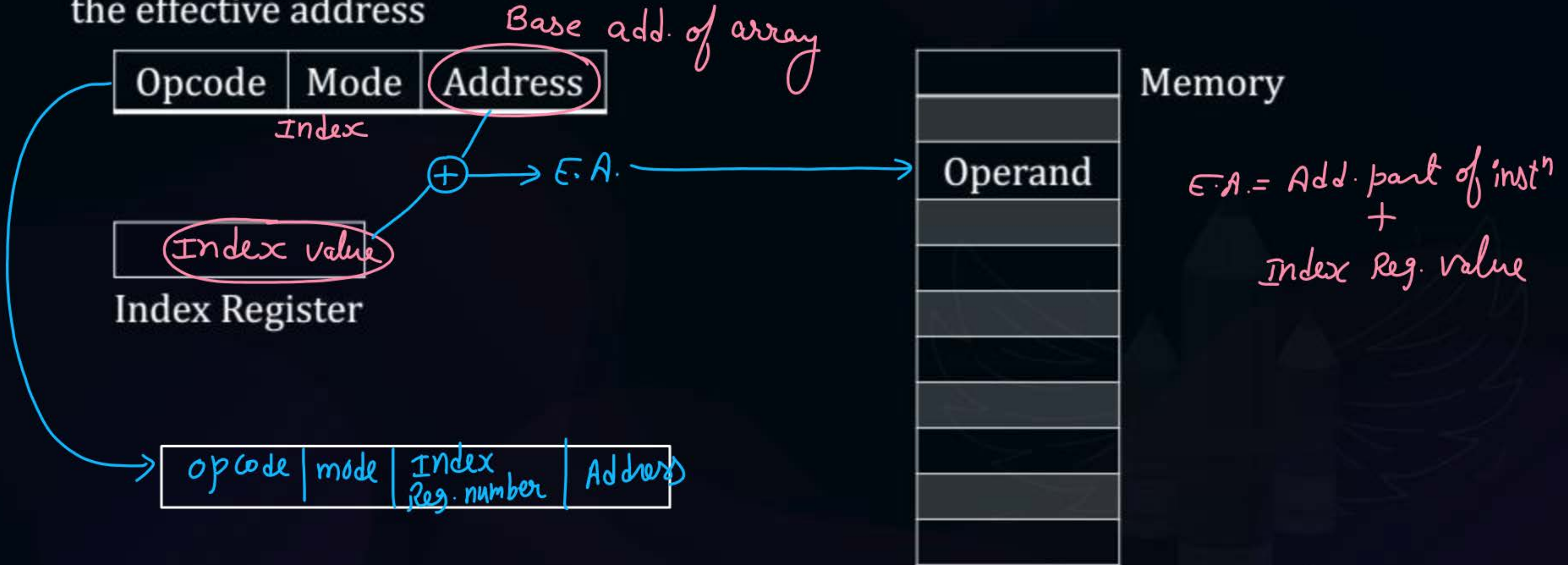


Topic : Indexed Mode

(Index Reg. mode)

Usage :- To access an element of array
or a member of structure

Address part of instruction (base address) is added to index register value to get the effective address

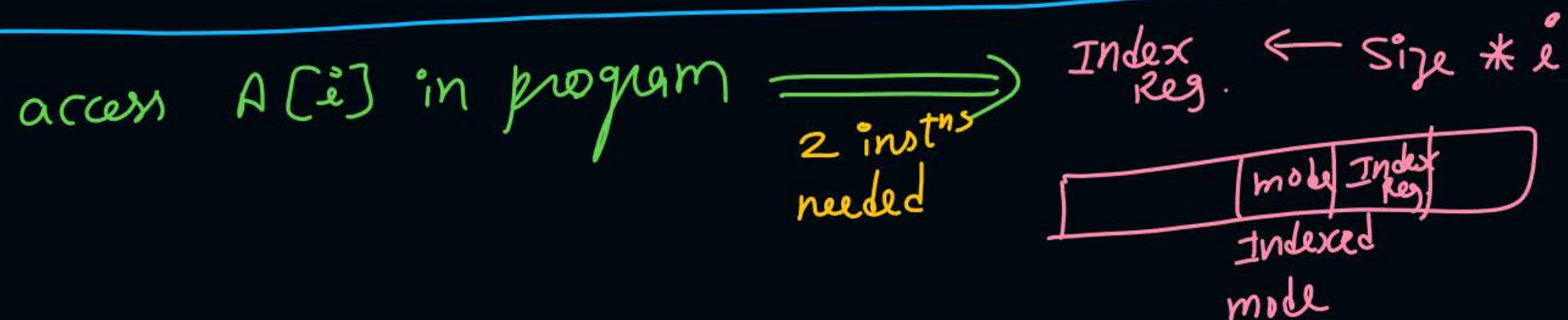


$$\text{add. of } A[i] = \text{Base} + \text{Size} * i$$

↓
mentioned
in instⁿ

↓
index
which is
maintained inside
index Reg.

Size = size of each element
in mem.



$$R1 \leftarrow M[R2 + 1000]$$

Assembly statement for indexed mode:-

mov R1, 1000(R2)

or

mov R1, R2(1000)

↑
Reg.
mode

Indexed mode

1000 $\Rightarrow$ base add.

R2 $\Rightarrow$ Index Reg.

$$E.A. = 1000 + R2$$

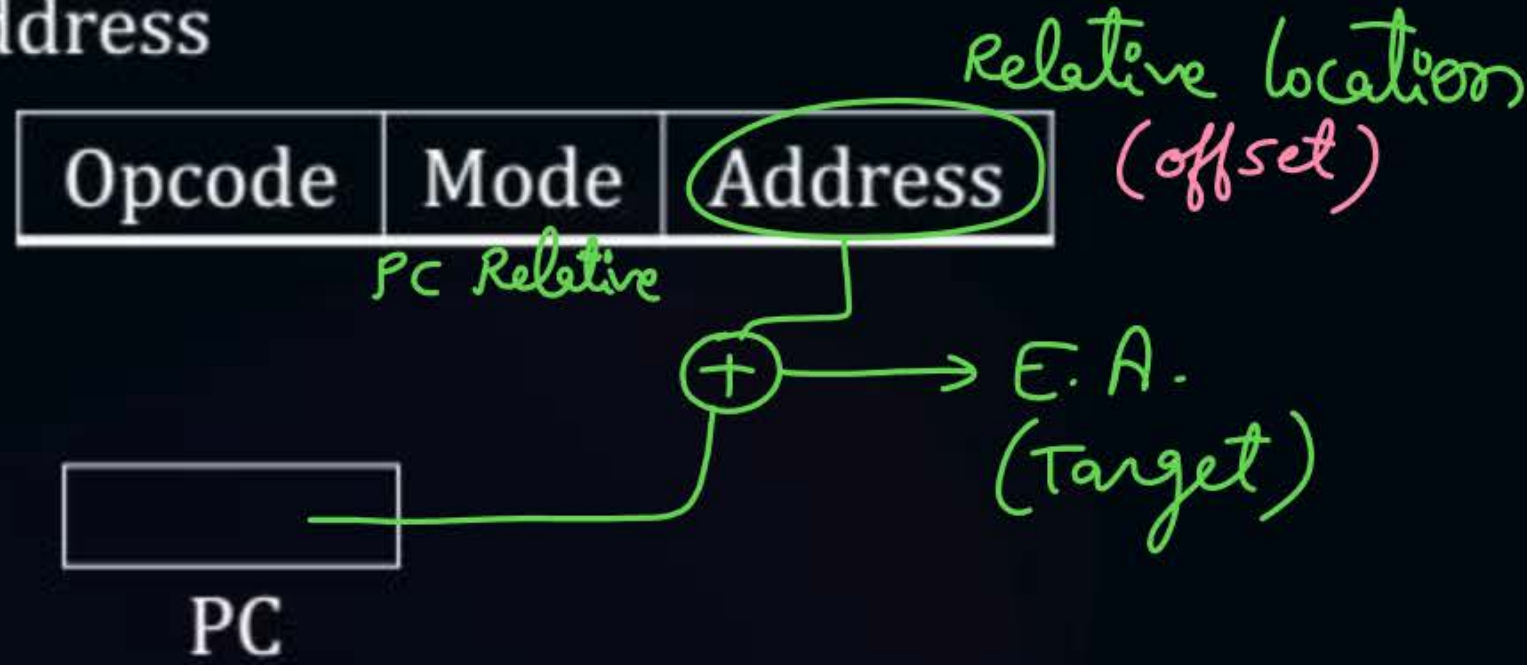


Topic : PC-Relative Mode

Position independent mode

used for intra segment branching

Address part of instruction (offset) is added to PC register value to get the effective address



Memory

	offset
forward jump	+ve
Backward jump	-ve

$$E.A. = PC \text{ value} + \text{address part of inst}^n$$

Assum CPU is executing
instⁿ I2.
PC = 504

CPU decodes I2 as branch
instⁿ

assume target of I2
is I7.
Target add. = 512

mem.

500	I1
502	I2
504	I3
506	I4
508	I5
510	I6
512	I7
514	I8
516	I9
...	...

Program
1 instⁿ $\Rightarrow$ 2 bytes

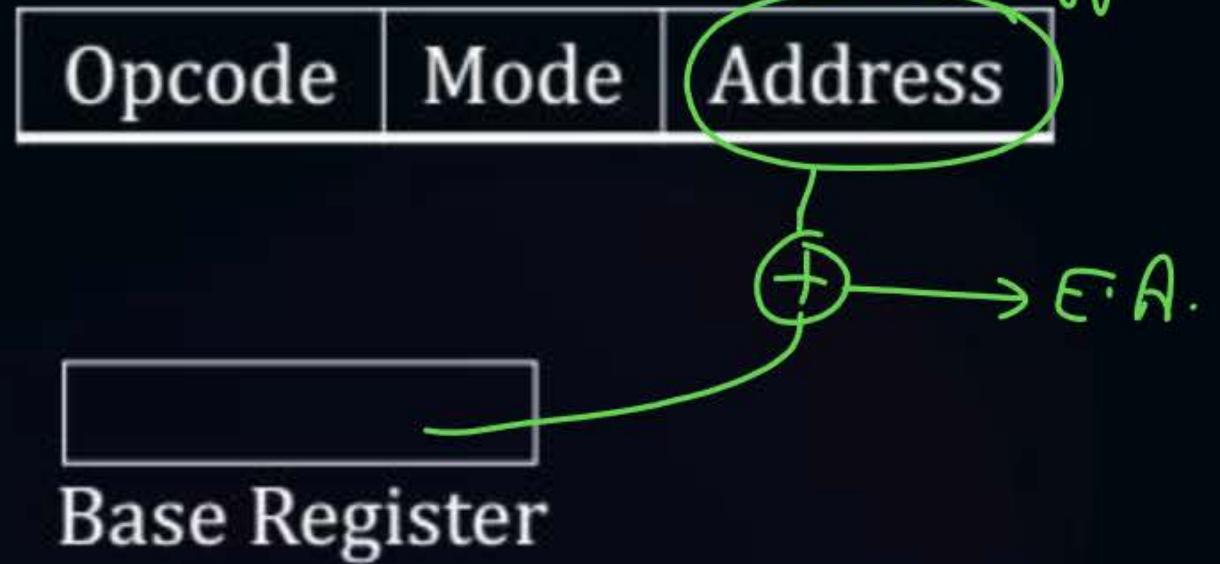
$$\text{Target add.} = \underbrace{504}_{\substack{\uparrow \\ \text{PC}}} + \textcircled{8} \quad \text{no. of relative mem. location to skip}$$



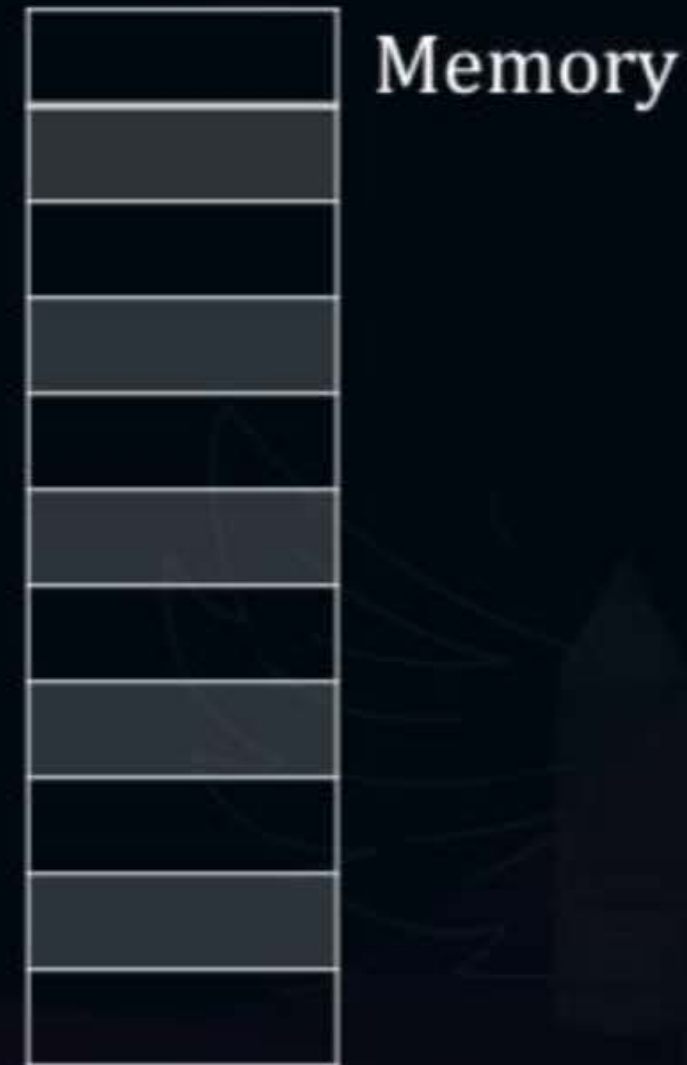
Topic : Base Register Mode

→ inter-segment branching

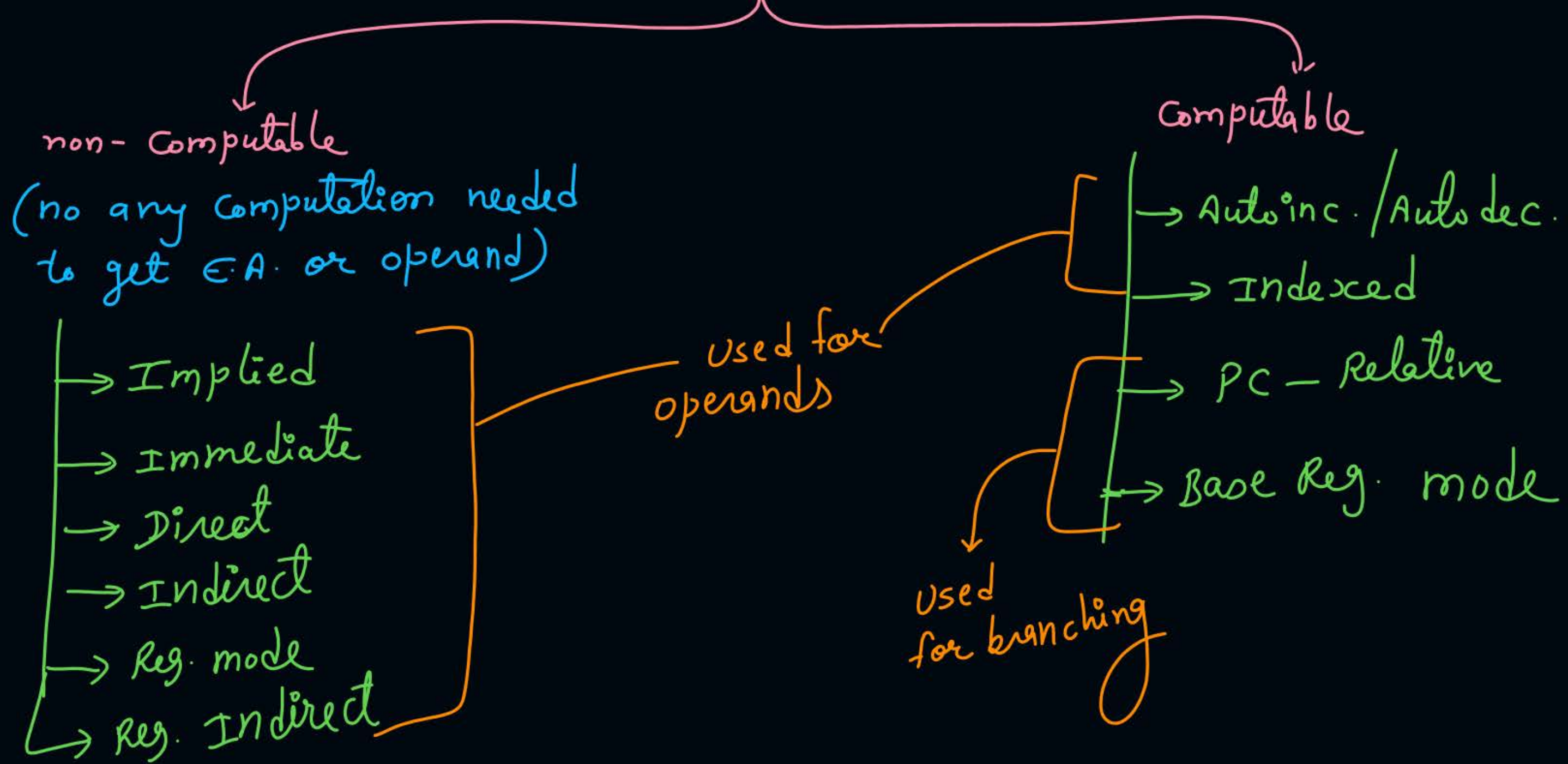
Address part of instruction (offset) is added to Base register value to get the effective address



$$E.A. = \text{Base Reg. value} + \text{Add. part of inst}^n$$



Types of add. modes





Topic : Example

opcode | mode | address
 | | = 500

mem. add.

Memory	
200	Opcode Mode
201	Address = 500
202	Next Instruction
...	
399	450
400	700
...	
500	800
...	
600	900
...	
702	-----
...	
800	300

202
PC = ~~200~~

R500
R500 = ~~400~~ 399

Index Reg.
XR = 100

AC

Mode	Effective Address	Operand
1. Immediate Mode	201	500
2. Direct Mode	500	800
3. Indirect Mode	800	300
4. Register Mode	—	400
5. Register Indirect Mode	400	700
6. Autodecrement Mode (Pre)	399	450
7. Indexed Mode	$500 + 100 = 600$	900
8. PC- Relative Mode	$500 + 202 = 702$	—



2 mins Summary



Topic

Addressing Modes

Topic

PC Relative Mode

Topic

Questions on Addressing Modes



Happy Learning

THANK - YOU

